Важно!
Чтобы продолжать учиться и учить

Подробнее

Урок 4. Цикл `while` — учимся крутиться как белка в колесе. Методический материал.

- Общая информация об уроке
- Структура урока
- Условие входа в цикл while
- ✅ Задача «Покормите котика»
- Бесконечный цикл
- Сокращённые операторы присваивания
- ✅ Задача «Белка в колесе»
- ✅ Задача «Печеньки»
- Моржовый оператор (:=)
- ✅ Задача «Ожидание»
- Множественное присваивание
- Поиск максимума/минимума
- ✅ Задача «Привет, Евклид!»
- ✅ Задача «Жара»
- ✅ Задача «Самый старший котик»
- ✅ Задача «Покормим всех»
- while + match – case
- Завершение урока

Общая информация об уроке

Тип урока: Новая тема
Предварительные знания учеников: Переменные, типы данных, арифметические операции, преобразование типов, ввод и вывод данных, f-строки, условный оператор `if – elif – else`, логические операторы, `match – case` (уроки 1 – 3)

На этом уроке ученики научатся:

- Использовать цикл `while` для повторения блока кода
- Понимать термин «итерация» и условие выхода из цикла
- Создавать бесконечный цикл `while True` и знать, как его прервать
- Использовать сокращённые операторы присваивания (`+=`, `-=` и др.)
- Применять паттерн «сигнал остановки» для обработки неизвестного количества вводов
- Использовать моржовый оператор (`:=`) для компактной записи цикла
- Применять множественное присваивание и обмен значений переменных
- Реализовывать паттерн поиска максимума/минимума в цикле
- Комбинировать `while` с `match – case`

Структура урока

№	Блок	Ориентировочное время	Задачи
1	Условие входа в цикл `while`	8 – 10 мин	—
2	**Задача «Покормите котика»**	5 – 8 мин	Базовый `while`, строка как условие остановки
3	Бесконечный цикл	4 – 6 мин	—
4	Сокращённые операторы присваивания	4 – 6 мин	—
5	**Задача «Белка в колесе»**	5 – 8 мин	`while` с пустой строкой как стоп-сигналом, счётчик
6	**Задача «Печеньки»**	5 – 8 мин	Счётчик + накопленная сумма, условие выхода по числу
7	Моржовый оператор (`:=`)	4 – 6 мин	—

№	Блок	Ориентировочное время	Задачи
9	Множественное присваивание	4 – 6 мин	—
10	Поиск максимума/ минимума	5 – 8 мин	—
11	**Задача «Привет, Евклид!»**	8 – 10 мин	`while` до равенства, алгоритм вычитания
12	**Задача «Жара»**	5 – 8 мин	Поиск максимума + стоп-сигнал
13	**Задача «Самый старший котик»**	8 – 10 мин	Поиск максимума с сохранением имени
14	**Задача «Покормим всех»**	5 – 8 мин	Поиск минимума + стоп-сигнал
15	`while` + `match – case`	4 – 6 мин	—
16	Подведение итогов	2–3 мин	—

⏱ **Темп урока:** Материала много — цикл `while` даётся ученикам тяжелее условного оператора, потому что здесь важно отслеживать состояние программы во времени. Задачи «Самый старший котик» и «Привет, Евклид!» могут стать буфером для быстрых учеников или перейти в домашнее задание. Следите за темпом группы и не торопитесь с первым блоком — он фундаментальный.

Условие входа в цикл `while`

Время: 8 – 10 минут. Центральная тема урока — не торопитесь, остальные блоки опираются на этот фундамент.

Ключевые идеи для объяснения

- До этого момента программы выполнялись линейно или разветвлялись. `while` — первый инструмент, который позволяет **повторять** действия. Связь с предыдущей темой: синтаксис `while` очень похож на `if`.
- Метафора из учебника хорошо работает: `while` — это Сизиф, который катит камень в гору, пока не достигнет вершины. Также работает метафора «котик

- После `while` **обязательно условие и двоеточие** (как и в `if`)
- Тело цикла — с **отступом в 4 пробела** (как и в `if`)
- После окончания тела цикла выполнение возвращается к проверке условия

```
got_results = False

while not got_results:
    print("🔄 Проверяю почту...")
    print("Результаты пришли?")
    if input() == 'да':
        got_results = True

print("📩 Письмо получено!")
```

- Обратите особое внимание: строка `print("📩 Письмо получено!")` стоит **без отступа** — она выполнится только после того, как цикл завершится.
- Один оборот тела цикла — **итерация**. Познакомьте учеников с этим термином сразу, он будет нужен на следующих уроках.
- Важное свойство: **если условие изначально ложно — цикл не выполнится ни разу**.

Рекомендация: Напишите пример живьём и «проиграйте» выполнение программы по шагам вслух: «Проверяем условие — оно истинно? Да — заходим в тело. Выполнили тело — снова проверяем условие...». Это помогает ученикам понять, что выполнение программы — это процесс, а не одномоментный результат.
Сравните `while` и `if` визуально на доске: у `if` стрелка идёт вниз, у `while` — стрелка возвращается назад. Это наглядно объясняет разницу.
Условие цикла записывается так же, как и для `if`: с помощью операторов отношения, логических операторов, булевых переменных или неявного приведения типов.

✅ Задача «Покормите котика»

Классная **Что проверяется:** базовая конструкция `while` с условием на строку; логика «повторяй, пока не получишь нужный ввод»; вывод на каждой итерации.

- Ключевой вопрос перед решением: «Что является условием продолжения цикла?» — пока введённая строка **не равна** `'Пошли кушать!'`.
- Паттерн «считать строку — проверить — повторить» — классика `while`. Разберите вместе до написания кода.
- Типичная ошибка: вынести `input()` только в заголовок цикла и потерять первый ввод, или, наоборот, забыть считывать новый ввод внутри тела.
- Обратите внимание: «Мяу!» выводится за **каждый неподходящий ввод**, но не после «Пошли кушать!». Проверьте граничный случай: если пользователь вводит правильную строку сразу — вывода нет.

Бесконечный цикл

Время: 4 – 6 минут.

Ключевые идеи для объяснения

- Если условие цикла **всегда истинно** — цикл никогда не завершится. Это называется **бесконечный цикл**.

```
while True:
    print("♾️ Бесконечность — не предел!")
```

- Запустите этот код на демонстрации — пусть ученики увидят бесконечный вывод своими глазами.
- Остановить бесконечный цикл в консоли: **`Ctrl + C`** — это прерывание процесса.
- **Практическое применение:** игры, веб-приложения, боты — они работают в бесконечном цикле и ждут событий. Это не баг, а архитектурное решение.

Рекомендация: После демонстрации бесконечного цикла сразу спросите: «Как сделать, чтобы цикл когда-нибудь завершился?» — это мотивирует следующий блок о сокращённых операторах и сигнале остановки. Напомните: всегда нужно проверять, есть ли в цикле условие для выхода.

Сокращённые операторы присваивания

Ключевые идеи для объяснения

- В циклах часто нужно **обновлять переменную** на каждой итерации: прибавить 1 к счётчику, отнять съеденные печеньки и т. д.
- Полная запись и сокращённая делают одно и то же:

```
counter = 0
counter = counter + 1  # Полная запись
counter += 1           # Сокращённая запись
```

- Все арифметические операторы поддерживают сокращённую форму: +=, -=, *=, /=, //=, **=, %=

```
cookie_count = 10
while cookie_count > 0:
    print("Съел печеньку. Ом-ном-ном.")
    cookie_count -= 1
print("Теперь я без печенек.")
```

- Обратите внимание на две обязательных вещи в цикле с числовым счётчиком:
    1. **Задать начальное значение** переменной до цикла.
    2. **Изменять переменную в теле цикла**, иначе цикл станет бесконечным.

Рекомендация: Намеренно «забудьте» изменить переменную в теле цикла на демонстрации — покажите, что получается бесконечный цикл. Пусть ученики сами объяснят, почему это происходит. Сокращённые операторы лучше сразу вводить как стандарт записи — длинную форму использовать только для объяснения смысла.

✅ Задача «Белка в колесе»

Классная **Что проверяется:** `while` со стоп-сигналом в виде **пустой строки**; счётчик итераций.

Как вести на уроке:

- Пустая строка как стоп-сигнал: `while line != "":` или короче — `while line:` (пустая строка приводится к `False`). Оба варианта верны, покажите оба.
- Счётчик: завести переменную `count = 0` до цикла, увеличивать `count += 1` на каждой итерации.
- Типичная ошибка: учесть пустую строку в счётчике. Напомните: стоп-сигнал не является содержательным значением и считать его не нужно.

✅ Задача «Печеньки»

Классная **Что проверяется:** `while` с условием на накопленную сумму; счётчик количества итераций; `int(input())` в теле цикла.

Как вести на уроке:

- Разберите вместе условие на доске: заканчиваем, когда **сумма >= вместимость желудка**. Это другой тип условия выхода — не по конкретному значению, а по достижению порога.
- Нужны две переменные: `total` (накопленная сумма) и `count` (количество забросов).
- Типичная ошибка: обновить счётчик после проверки (тогда последняя горсть, переполнившая желудок, не считается). Разберите, когда именно нужно увеличивать счётчик.
- Граничный случай: первая же горсть превышает вместимость — ответ 1.

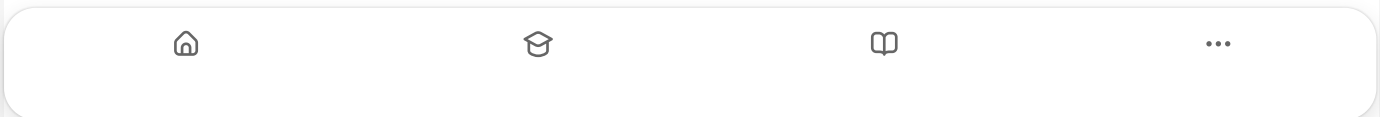
Моржовый оператор (:=)

Время: 4 – 6 минут.

Ключевые идеи для объяснения

- Оператор `:=` позволяет **присвоить значение переменной прямо в условии** `while`. Это убирает дублирование кода, когда нужно и присвоить, и сразу проверить.

Сравните:



```
while text != "quit":
    print(f"Ты написал: {text}")
    text = input()

# С моржом — input() один раз
while (text := input()) != "quit":
    print(f"Ты написал: {text}")
```

- **Скобки обязательны:** они повышают приоритет присваивания. Без скобок сначала выполнится сравнение, и в переменную запишется `True`/`False`, а не введённая строка.
- Ещё пример — проверка длины пароля:

```
while len(text := input("Введите пароль:\n")) < 8:
    print("Пароль слишком короткий!")
```

Рекомендация: Моржовый оператор — не обязательный инструмент, но очень частый в реальном коде. Объясните его как «синтаксический сахар» — программа работает одинаково, просто запись компактнее. Если кому-то непонятно — можно всегда вернуться к «длинной» форме. Акцент: зачем скобки? Разберите приоритет вычислений вслух.

✅ Задача «Ожидание»

Классная **Что проверяется:** `while` с условием выхода по одному из двух значений; счётчик пропущенных автобусов; считывание двух чисел-подсказок до цикла.

Как вести на уроке:

- Разберите структуру задачи: сначала вводим два «нужных» числа, потом в цикле считываем числа до тех пор, пока не встретим одно из них.
- Условие цикла: `while bus != a and bus != b` — оба условия через `and`, потому что нужно, чтобы введённое число не совпадало **ни с одним** из двух.
- Счётчик увеличивается только для **не подходящих** автобусов — то есть внутри тела цикла.
- Если применить моржовый оператор: `while (bus := int(input())) != a and bus != b:` — это хорошая иллюстрация его пользы.

Множественное присваивание

Время: 4 – 6 минут.

Ключевые идеи для объяснения

- Python позволяет присвоить значения нескольким переменным **в одну строку**:

```
cat_name, dog_name = "Барсик", "Шарик"
```

- Главное применение в контексте этого урока — **обмен значениями** без вспомогательной переменной:

```
# Классика через третью переменную
temp = a
a = b
b = temp

# Магия Python
a, b = b, a
```

- Можно жонглировать тремя и более переменными:

```
x, y, z = "огонь", "вода", "земля"
x, y, z = z, x, y
```

Рекомендация: Спросите у класса: «Как поменять значения двух стаканов с жидкостью?» — нужен третий стакан. Python это обходит. Покажите пример с обменом — он часто встречается в задачах на сортировку, к которым ученики придут позже. Пример с игрой в напёрстки из учебника хорошо иллюстрирует обмен в цикле.

Ключевые идеи для объяснения

- Паттерн поиска максимума или минимума — типовая задача, которая встречается постоянно.
- Алгоритм: заводим переменную с заведомо «проигрышным» начальным значением, затем на каждой итерации обновляем её, если текущий элемент «лучше».

```
biggest_book = 0              # Любая книга > 0 страниц
smallest_book = float("+inf")  # Любая книга < бесконечности

while (n := int(input())) > 0:
    if n > biggest_book:
        biggest_book = n
    smallest_book = min(smallest_book, n)

print(biggest_book)
print(smallest_book)
```

- Альтернатива: использовать **первый элемент** как начальный максимум/минимум. Это работает, если гарантированно будет хотя бы один ввод.
- Встроенные функции `min()` и `max()` можно использовать вместо `if` для обновления — покажите оба варианта.

Рекомендация: Разыграйте алгоритм в классе: попросите нескольких учеников называть числа, а вы «держите» текущий максимум и обновляете его вслух. Когда принцип понят «руками» — переходите к коду. Акцент на выборе начального значения: почему именно `0` или `float("+inf")`, а не произвольное?

✅ Задача «Привет, Евклид!»

Классная **Что проверяется:** `while` с условием на равенство двух переменных; алгоритм вычитания (НОД по Евклиду); понимание инварианта цикла.

Как вести на уроке:

- Структура цикла: `while a != b:` — внутри `if a > b: a -= b` иначе `b -= a`.
- Типичная ошибка: написать `while a != b` и забыть обновить переменные — бесконечный цикл.
- Граничный случай: если числа сразу равны — цикл не выполнится, вывести любое из них.
- Хорошая задача для обсуждения: почему алгоритм всегда завершается? (На каждом шаге хотя бы одно число уменьшается.)

✅ Задача «Жара»

Классная **Что проверяется:** поиск максимума в цикле `while`; стоп-сигнал — конкретное число (6000); стоп-сигнал не включается в поиск.

Как вести на уроке:

- Ключевой момент: 6000 — это **признак конца ввода**, его не нужно учитывать при поиске максимума. Разберите это явно.
- Удобный паттерн с моржовым оператором: `while (t := int(input())) != 6000:` — сразу и считывает, и проверяет.
- Начальное значение для максимума: можно взять `-float("inf")` или первый элемент (но тогда структура цикла немного другая). Обсудите оба подхода.
- Типичная ошибка: взять начальное значение максимума 0 и получить неправильный ответ при отрицательных температурах (что вполне реально!).

✅ Задача «Самый старший котик»

Классная / Домашняя **Что проверяется:** ввод пары «имя + число» на каждой итерации; поиск максимального числа с сохранением соответствующей строки; стоп-сигнал — отрицательный возраст; первый среди равных.

Как вести на уроке:

- Разберите структуру ввода: на каждой итерации сначала вводится имя, потом возраст. Если возраст отрицательный — стоп, имя в этот момент не нужно.

- Условие «первый среди равных»: использовать строгое > (не >=) при обновлении максимума.
- Паттерн: считать имя → считать возраст → проверить стоп → обновить максимум.
- Типичная ошибка: перепутать порядок ввода или не инициализировать `best_age` до цикла.

✅ Задача «Покормим всех»

Классная **Что проверяется:** поиск минимума в цикле `while`; стоп-сигнал — отрицательное число; корректное начальное значение для минимума.

Как вести на уроке:

- Полная аналогия с задачей «Жара», только ищем минимум. Хороший момент, чтобы ученики самостоятельно предложили решение по аналогии.
- Начальное значение для минимума: `float("+inf")` или первый элемент.
- Стоп-сигнал не считается: отрицательное число — признак конца, а не реальное количество еды.
- Типичная ошибка: взять `0` как начальный минимум — тогда ответ будет 0, если все введённые числа положительные.

`while` + `match - case`

Время: 4 – 6 минут. Блок обзорный — показывает, что блочные операторы комбинируются.

Ключевые идеи для объяснения

- В теле `while` может стоять любой другой блочный оператор — в том числе `match - case`. Это уже видели с `if`, здесь принцип тот же.
- Классическое применение: меню с выбором действия в игре или интерактивной программе.

```
health = 100
run_away = False
```



```
    print("Ваш ход (атака/защита/бег):")
```

```
    match input():
        case "атака":
            print("⚔️ Атакую врага!")
            health -= 10
        case "защита":
            print("🛡️ Защищаюсь!")
            health += 5
        case "бег":
            print("🏃 Бегу!")
            run_away = True
        case _:
            print("🤷 Неизвестная команда.")
    print(f"❤️ Здоровье: {health}")

if health <= 0:
    print("💀 Игра окончена.")
else:
    print("🏆 Победа!")
```

Рекомендация: Запустите этот пример и сыграйте вместе с классом — это живо демонстрирует, как `while` и `match-case` работают в связке. Акцент: условие цикла составное (`and`), а внутри тела — отдельный `match – case`. Это хороший момент напомнить о вложенных блочных операторах.

Завершение урока

Время: 2–3 минуты.

Что обязательно сказать

- Сегодня программы научились повторять действия — это делает их по-настоящему мощными. Почти ни одна реальная программа не обходится без циклов.
- Напомнить про **классные и домашние задачи**: если что-то не успели — продолжить дома.
- **Домашнее задание:** дочитать урок в учебнике, выполнить домашние задачи в ЛМС.
- Напомнить, где задавать вопросы: чат задачи в ЛМС и групповой чат.

Таблица задач урока

Классная работа

Задача	Баллы	Что проверяется	На что обратить особое внимание
Покормите котика	10	Базовый `while` со строкой как условием выхода; вывод на каждой итерации	Не забыть считывать новый ввод внутри тела цикла. Граничный случай: правильная строка с первого раза — вывода нет
Белка в колесе	10	`while` с пустой строкой как стоп-сигналом; счётчик итераций	Пустая строка как стоп-сигнал не считается. Можно использовать `while line:` вместо `while line != ""`
Печеньки	10	`while` с условием на накопленную сумму; счётчик забросов	Когда именно увеличивать счётчик (до или после проверки). Граничный случай: первая горсть превышает желудок
Ожидание	10	`while` до одного из двух значений; счётчик пропущенных; `and` в условии	`and`, а не `or` в условии. Моржовый оператор уместен здесь. Типичная ошибка — перепутать логический оператор
Привет, Евклид!	15	`while` с условием на равенство двух переменных; алгоритм вычитания	На каждом шаге одна из переменных уменьшается — цикл конечен. Граничный случай: числа уже равны
Жара	15	Поиск максимума в `while`; стоп-сигнал 6000 не учитывается	Начальное значение максимума должно допускать отрицательные температуры. Стоп-сигнал не включать в поиск
Самый старший котик	15	Поиск максимума с сохранением связанного значения (имени); ввод пары на итерации; первый среди	Строгое `>` для «первого среди равных». Правильная инициализация `best_age` до

Задача	Баллы	Что проверяется	На что обратить особое внимание
Покормим всех	15	Поиск минимума; стоп-сигнал — отрицательное число	Начальное значение `float("+inf")`, а не `0`. Стоп-сигнал не включать в поиск

Домашние задачи

Задача	Баллы	Что проверяется	На что обратить особое внимание
Катим вверх	20	`while` с изменяемым состоянием (позиция); вывод на каждой итерации; остановка при достижении вершины	Корректное обновление позиции: отрицательные числа уменьшают расстояние до вершины. Вывести «Наконец-то!» в момент достижения и прекратить ввод
Черёмуховые холода	20	`while` с пустой строкой как стоп-сигналом; `in` для фильтрации строк; поиск лексикографического минимума среди подходящих	Учитывать только строки, содержащие заданное слово. Начальное значение минимума — пустая строка или `None` с обработкой первого совпадения
Краткость — сестра таланта	20	`while` с `in` как стоп-сигналом (`'end'` в строке); поиск минимальной длины; последняя среди равных	Стоп-сигнал — вхождение `'end'` в строку (не равенство). «Последняя среди равных» — использовать `<=`, а не `<` при обновлении
Дюна	20	`while` со стоп-сигналом «число < 10»; поиск максимума среди чётных чисел; фильтрация нечётных	Стоп-сигнал не включается в поиск. Проверить чётность: `n % 2 == 0`. Начальное значение для максимума среди чётных
Похвалите котика	20	`while` с `in` как стоп-сигналом (`'scat'` или `'Scat'`); подсчёт количества строк и суммы их	Стоп-строка не включается в подсчёт. `round(result, 5)` для округления. Граничный случай: